

Attention Mechanism Deep Dive

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

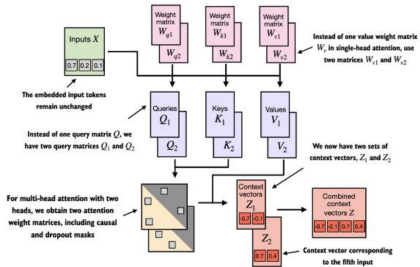


جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 18, 2026

1. Motivation for Attention Mechanisms
2. Learning Outcomes
3. Alignment Function
4. Query, Key, Value (QKV)
5. Attention-based Decoding Sequences
6. Summary
7. References



- ▶ Sequence-to-sequence models struggle with **long-range dependencies**, making it difficult to capture context over lengthy inputs.
- ▶ Attention mechanisms address this by allowing the model to **selectively focus** on relevant parts of the input sequence during processing.
- ▶ They form the **backbone** of modern architectures such as **Transformer models** (e.g., BERT, GPT, ViT).
- ▶ Attention is crucial in various tasks including **translation**, **summarization**, and **image captioning**.

Example: In machine translation, rather than encoding an entire sentence into a single fixed-size vector, attention enables the model to dynamically focus on relevant source words when generating each target word.

After this session, you should be able to:

- ▶ Understand how **alignment functions** operate.
- ▶ Comprehend **QKV (Query, Key, Value)** mechanism in attention.
- ▶ Learn how attention is used in **decoding sequences**.
- ▶ Recognize the **strengths and limitations** of attention mechanisms.
- ▶ Get familiar with **modern attention research**.

- ▶ The alignment function is a crucial component of attention mechanisms, determining how much focus to place on different parts of the input sequence.
- ▶ It computes a score for each input token relative to the current output token being generated.
- ▶ The scores are then normalized (often using softmax) to create a probability distribution, indicating the relative importance of each input token.

Common alignment functions include:

- ▶ **Dot Product:** Computes the dot product between the query and key vectors.
- ▶ **Scaled Dot Product:** Similar to dot product but scaled by the square root of the dimension of the key vectors to prevent large values that can saturate the softmax function.
- ▶ **Additive Attention:** Combines the query and key vectors using a feed-forward neural network to compute the alignment scores.

The choice of alignment function can significantly impact the model's performance, especially in tasks with long sequences or complex dependencies.

Example: In a translation task, the alignment function helps the model determine which words in the source language should be emphasized when generating each word in the target language. For instance, when translating “The cat sat on the mat,” the model might focus more on “cat” when generating “gato” in Spanish, while still considering “sat” and “on” to maintain the context.

► **Dot Product:**

$$\text{score} = \mathbf{q}^\top \mathbf{k}$$

► **Scaled Dot Product:**

$$\text{score} = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$$

(used in Transformers to prevent softmax saturation)

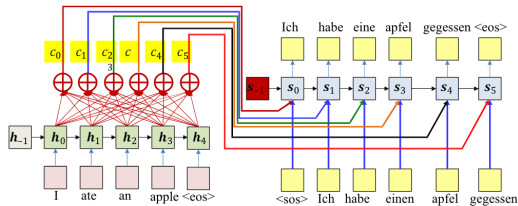
► **Additive (Bahdanau) Attention:**

$$\text{score} = \mathbf{v}^\top \tanh(W_1 h_1 + W_2 s_t)$$

► **General:**

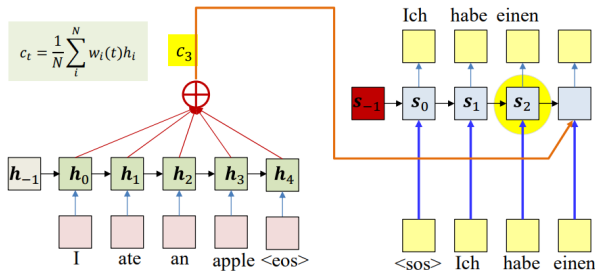
$$\text{score} = \mathbf{q}^\top W_k \mathbf{k}$$

Each function balances efficiency vs expressiveness.



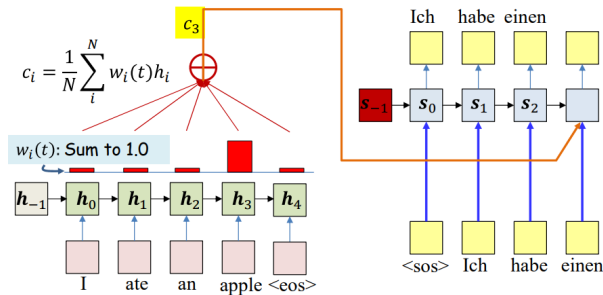
$$c_t = \frac{1}{N} \sum_{i=1}^N w_i(t) h_i$$

- **Attention weights:** The weights $w_i(t)$ are dynamically computed as functions of decoder state
 - Expectation: if the model is well-trained, this will automatically “highlight” the correct input
- But how are these computed?

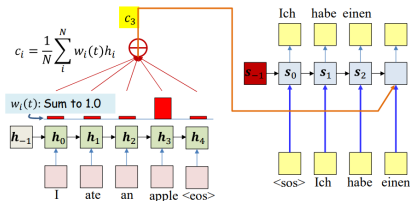


- ▶ The “attention” weights $w_i(t)$ must be computed from available information at time t .
- ▶ The primary information is s_{t-1} (the state at time $t - 1$).
- ▶ Also, the input word at time t , but generally not used for simplicity.

$$c_t = \sum_{i=1}^N w_i(t) h_i$$



- ▶ The weights must be positive and sum to 1.0 (i.e., be a distribution).
- ▶ Ideally, they must be high for the most relevant inputs for the i -th output and low elsewhere.
- ▶ **Solution: A two step weight computation**
 - First compute raw weights (which could be +ve or -ve).
 - Then softmax them to convert them to a distribution.



- The weights $w_i(t)$ must be positive and sum to 1.0
 - I.e. be a distribution
 - Ideally, they must be high for the most relevant inputs for the i th output and low elsewhere
- Solution: A two step weight computation
 - First compute raw weights (which could be +ve or -ve)
 - Then softmax them to convert them to a distribution

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

- ★ The attention framework computes a different “context” vector at each output step (T/F)
- ◆ True
 - ◆ False

★ The attention framework computes a different “context” vector at each output step (T/F)

◆ True

◆ False

- ★ The context vector is chosen as the hidden (encoder) representation of the input word that is assigned the highest attention weight (T/F)
- ◆ True
 - ◆ False

- ★ The context vector is chosen as the hidden (encoder) representation of the input word that is assigned the highest attention weight (T/F)

◆ True

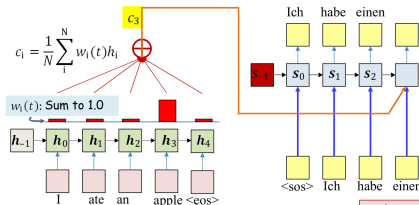
◆ False

- ★ The attention weight to any input word is a function of the hidden encoder representation of the word and the most recent decoder state (T/F)
- ◆ True
 - ◆ False

- ★ The attention weight to any input word is a function of the hidden encoder representation of the word and the most recent decoder state (T/F)

◆ True

◆ False

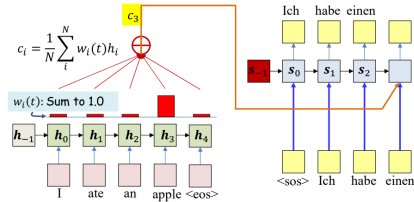


- The weights $w_i(t)$ must be positive and sum to 1.0
 - i.e. be a distribution
 - Ideally, they must be high for the most relevant inputs for the i th output and low elsewhere
- Solution: A two step weight computation
 - First compute *raw* weights (which could be +ve or -ve)
 - Then softmax them to convert them to a distribution

What is this function?

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$



- Typical options for $g()$ (variables in red must be learned)

$$g(h_i, s_{t-1}) = h_i^T s_{t-1}$$

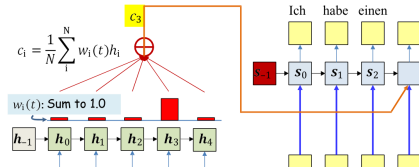
$$g(h_i, s_{t-1}) = h_i^T \mathbf{w}_g s_{t-1}$$

$$g(h_i, s_{t-1}) = \mathbf{v}_g^T \tanh\left(\mathbf{w}_g \begin{bmatrix} h_i \\ s_{t-1} \end{bmatrix}\right)$$

$$g(h_i, s_{t-1}) = \text{MLP}([h_i, s_{t-1}])$$

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$



Let's consider a typical conversion process

assuming this model as an example

- typical options for $g()$ (variables in red must be learned)

$$g(h_i, s_{t-1}) = h_i^T s_{t-1}$$

$$g(h_i, s_{t-1}) = h_i^T \mathbf{w}_g s_{t-1}$$

$$g(h_i, s_{t-1}) = \mathbf{v}_g^T \tanh \left(\mathbf{w}_g \begin{bmatrix} h_i \\ s_{t-1} \end{bmatrix} \right)$$

$$g(h_i, s_{t-1}) = \text{MLP}([h_i, s_{t-1}])$$

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

- ▶ **Query:** What you want (the search)
- ▶ **Key:** Index of content (where to look)
- ▶ **Value:** Actual content (what you get)

Intuition

Think of QKV like a search engine:

- ▶ **Query (Q):** Comes from the current decoder state (or token)
- ▶ **Key (K), Value (V):** Come from the encoder (or input sequence)

Given:

- ▶ Input sequence $\mathbf{X}$

Linear projections:

$$\mathbf{Q} = \mathbf{X}W_Q$$

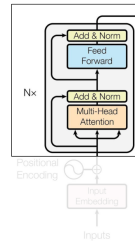
$$\mathbf{K} = \mathbf{X}W_K$$

$$\mathbf{V} = \mathbf{X}W_V$$

Attention:

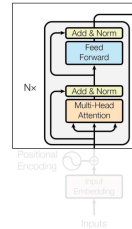
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Query
- Key
- Value



Database

{Key, Value store}

```
{
  "order_100": {
    "items": "a1",
    "delivery_date": "a2",
    ...
  },
  "order_101": {
    "items": "b1",
    "delivery_date": "b2",
    ...
  },
  "order_102": {
    "items": "c1",
    "delivery_date": "c2",
    ...
  },
  "order_103": {
    "items": "d1",
    "delivery_date": "d2",
    ...
  },
  "order_104": {
    "items": "e1",
    "delivery_date": "e2",
    ...
  },
  "order_105": {
    "items": "f1",
    "delivery_date": "f2",
    ...
  },
  "order_106": {
    "items": "g1",
    "delivery_date": "g2",
    ...
  },
  "order_107": {
    "items": "h1",
    "delivery_date": "h2",
    ...
  },
  "order_108": {
    "items": "i1",
    "delivery_date": "i2",
    ...
  },
  "order_109": {
    "items": "j1",
    "delivery_date": "j2",
    ...
  },
  "order_110": {
    "items": "k1",
    "delivery_date": "k2",
    ...
  }
}
```

Database
{Key, Value store}

{Query: "Order details of order_104"}
OR
{Query: "Order details of order_106"}

```
{
  "order_100": {
    "items": "a1",
    "delivery_date": "a2",
    ...
  },
  "order_101": {
    "items": "b1",
    "delivery_date": "b2",
    ...
  },
  "order_102": {
    "items": "c1",
    "delivery_date": "c2",
    ...
  },
  "order_103": {
    "items": "d1",
    "delivery_date": "d2",
    ...
  },
  "order_104": {
    "items": "e1",
    "delivery_date": "e2",
    ...
  },
  "order_105": {
    "items": "f1",
    "delivery_date": "f2",
    ...
  },
  "order_106": {
    "items": "g1",
    "delivery_date": "g2",
    ...
  },
  "order_107": {
    "items": "h1",
    "delivery_date": "h2",
    ...
  },
  "order_108": {
    "items": "i1",
    "delivery_date": "i2",
    ...
  },
  "order_109": {
    "items": "j1",
    "delivery_date": "j2",
    ...
  },
  "order_110": {
    "items": "k1",
    "delivery_date": "k2",
    ...
  }
}
```


{Key, Value store}

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{
  "order_100": {"items": "a1", "delivery_date": "a2", ...},
  "order_101": {"items": "b1", "delivery_date": "b2", ...},
  "order_102": {"items": "c1", "delivery_date": "c2", ...},
  "order_103": {"items": "d1", "delivery_date": "d2", ...},
  "order_104": {"items": "e1", "delivery_date": "e2", ...},
  "order_105": {"items": "f1", "delivery_date": "f2", ...},
  "order_106": {"items": "g1", "delivery_date": "g2", ...},
  "order_107": {"items": "h1", "delivery_date": "h2", ...},
  "order_108": {"items": "i1", "delivery_date": "i2", ...},
  "order_109": {"items": "j1", "delivery_date": "j2", ...},
  "order_110": {"items": "k1", "delivery_date": "k2", ...}
}
```

{Key, Value store}

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{
  "order_100": {"items": "a1", "delivery_date": "a2", "..."},
  "order_101": {"items": "b1", "delivery_date": "b2", "..."},
  "order_102": {"items": "c1", "delivery_date": "c2", "..."},
  "order_103": {"items": "d1", "delivery_date": "d2", "..."},
  "order_104": {"items": "e1", "delivery_date": "e2", "..."},
  "order_105": {"items": "f1", "delivery_date": "f2", "..."},
  "order_106": {"items": "g1", "delivery_date": "g2", "..."},
  "order_107": {"items": "h1", "delivery_date": "h2", "..."},
  "order_108": {"items": "i1", "delivery_date": "i2", "..."},
  "order_109": {"items": "j1", "delivery_date": "j2", "..."},
  "order_110": {"items": "k1", "delivery_date": "k2", "..."}
}
```

{Key, Value store}

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{
  "order_100": {"items": "a1", "delivery_date": "a2", ...},
  "order_101": {"items": "b1", "delivery_date": "b2", ...},
  "order_102": {"items": "c1", "delivery_date": "c2", ...},
  "order_103": {"items": "d1", "delivery_date": "d2", ...},
  "order_104": {"items": "e1", "delivery_date": "e2", ...},
  "order_105": {"items": "f1", "delivery_date": "f2", ...},
  "order_106": {"items": "g1", "delivery_date": "g2", ...},
  "order_107": {"items": "h1", "delivery_date": "h2", ...},
  "order_108": {"items": "i1", "delivery_date": "i2", ...},
  "order_109": {"items": "j1", "delivery_date": "j2", ...},
  "order_110": {"items": "k1", "delivery_date": "k2", ...}
}
```

Done at the same time !!

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

{Key, Value store}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... } },  
{ "order_101": { "items": "b1", "delivery_date": "b2", ... } },  
{ "order_102": { "items": "c1", "delivery_date": "c2", ... } },  
{ "order_103": { "items": "d1", "delivery_date": "d2", ... } },  
{ "order_104": { "items": "e1", "delivery_date": "e2", ... } },  
{ "order_105": { "items": "f1", "delivery_date": "f2", ... } },  
{ "order_106": { "items": "g1", "delivery_date": "g2", ... } },  
{ "order_107": { "items": "h1", "delivery_date": "h2", ... } },  
{ "order_108": { "items": "i1", "delivery_date": "i2", ... } },  
{ "order_109": { "items": "j1", "delivery_date": "j2", ... } },  
{ "order_110": { "items": "k1", "delivery_date": "k2", ... } }
```

(Query: "Order details of order_104")
OR
(Query: "Order details of order_106")

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... } },  
{ "order_101": { "items": "b1", "delivery_date": "b2", ... } },  
{ "order_102": { "items": "c1", "delivery_date": "c2", ... } },  
{ "order_103": { "items": "d1", "delivery_date": "d2", ... } },  
{ "order_104": { "items": "e1", "delivery_date": "e2", ... } },  
{ "order_105": { "items": "f1", "delivery_date": "f2", ... } },  
{ "order_106": { "items": "g1", "delivery_date": "g2", ... } },  
{ "order_107": { "items": "h1", "delivery_date": "h2", ... } },  
{ "order_108": { "items": "i1", "delivery_date": "i2", ... } },  
{ "order_109": { "items": "j1", "delivery_date": "j2", ... } },  
{ "order_110": { "items": "k1", "delivery_date": "k2", ... } }
```

Query
1. Search for
info

Key
1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most
relevant and by how much

Value
1. Actual details of the object
2. More fine grained

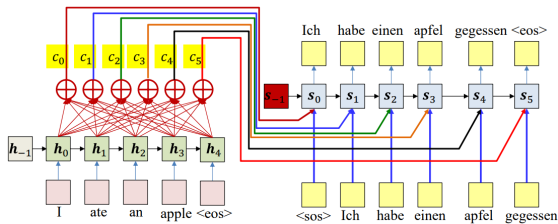
Attention-Based Decoding – Overview

- ▶ The decoder uses attention to select relevant encoder hidden states at each decoding step.

Combines:

- Previous decoder state
 - Encoder hidden states
 - Attention context vector
- ▶ The output at time t :

$$y_t = \text{Decoder}(y_{t-1}, \text{context}_t)$$



$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

$$c_t = \frac{1}{N} \sum_i^N w_i(t) h_i$$

1. **Compute query:** Generate a query vector from the current decoder hidden state.
2. **Align with encoder outputs:** Compare the query with encoder outputs (keys) to measure relevance.
3. **Get attention weights:** Apply softmax to the alignment scores to obtain attention weights.
4. **Compute context:** Calculate the context vector as the weighted sum of encoder outputs using the attention weights.
5. **Feed into decoder:** Use the context vector and previous outputs as input for the next decoding step.





Query



Key Value
Store



Key



Value

Done at the same time !!



Query



Key Value
Store



Key



Value

Parallelizable !!!



Query

Q



Key Value
Store

QK^T



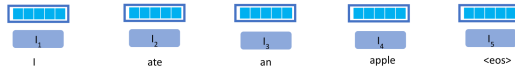
Key

$\text{softmax}(\frac{QK^T}{\sqrt{d}})$

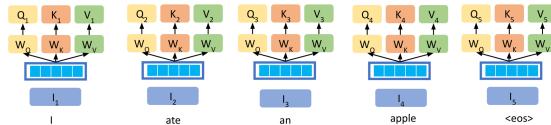


Value

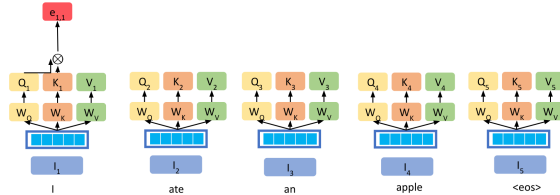
$\text{softmax}(\frac{QK^T}{\sqrt{d}})V$



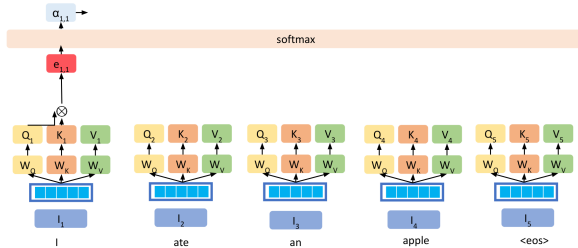
Dimensions across QKV have been dropped for brevity

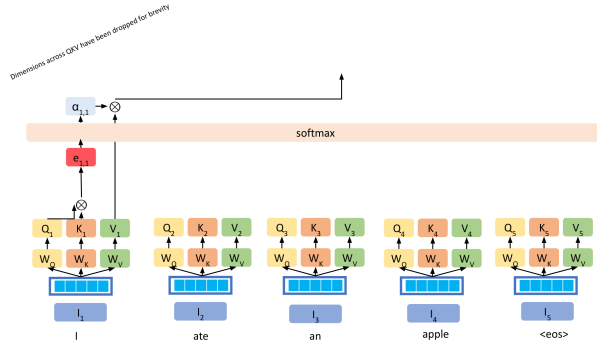


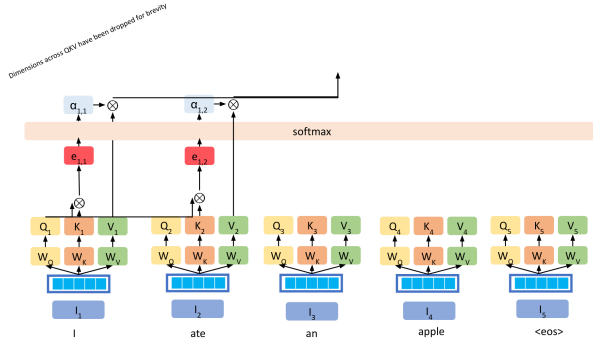
Dimensions across QKV have been dropped for brevity

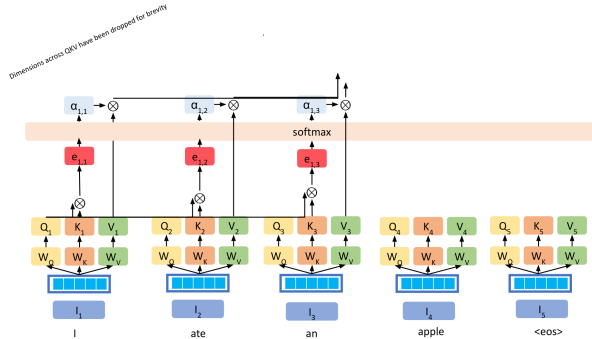


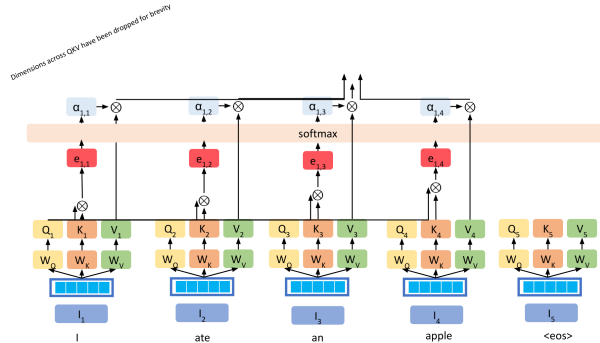
Dimensions across QKV have been dropped for brevity



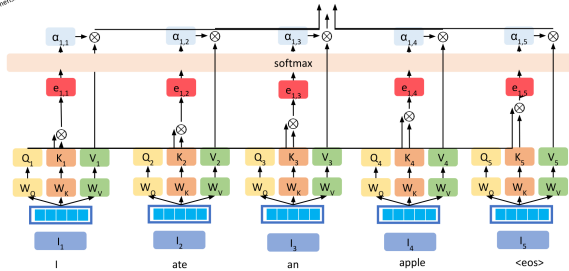


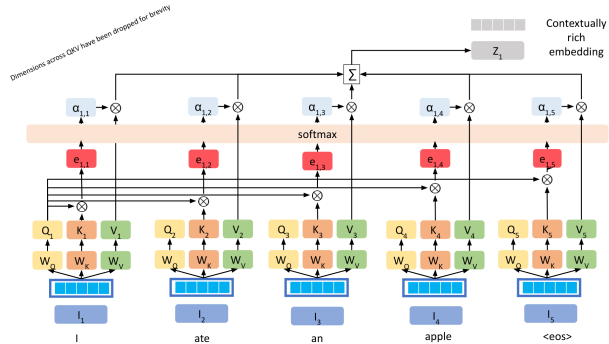


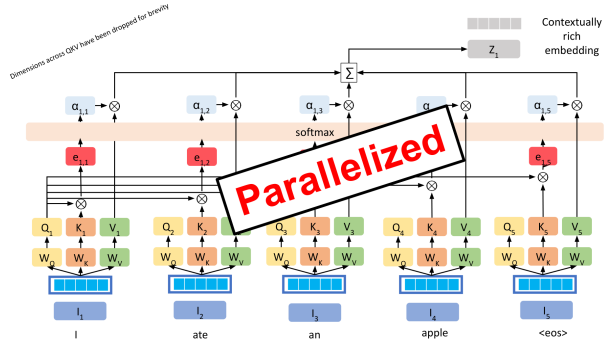


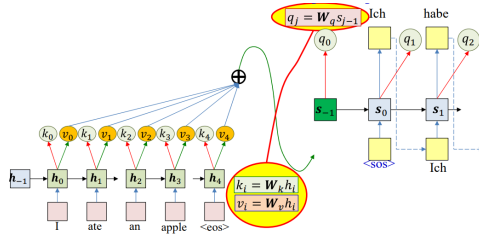


Dimensions across QKV have been dropped for brevity

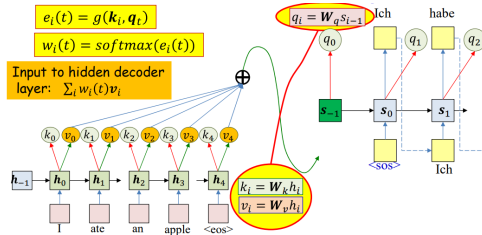




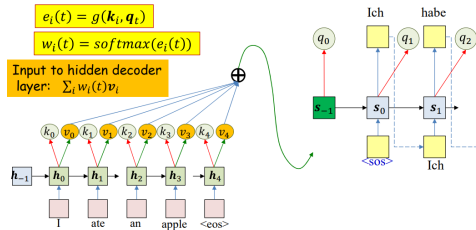




- Encoder outputs an explicit "key" and "value" at each input time
 - Key is used to evaluate the importance of the input at that time, for a given output
- Decoder outputs an explicit "query" at each output time
 - Query is used to evaluate which inputs to pay attention to
- The weight is a function of key and query
- The actual context is a weighted sum of value



- Encoder outputs an explicit "key" and "value" at each input time
 - Key is used to evaluate the importance of the input at that time, for a given output
- Decoder outputs an explicit "query" at each output time
 - Query is used to evaluate which inputs to pay attention to
- The weight is a function of key and query
- The actual context is a weighted sum of value



Special case: $k_i = v_i = h_i$

$$q_t = s_{t-1}$$

We will continue using this assumption in the following slides but in practice the query-value format is used

```
# Assuming encoded input
# (K,V) = [ $\mathbf{k}_{enc}[0] \dots \mathbf{k}_{enc}[T]$ ], [ $\mathbf{v}_{enc}[0] \dots \mathbf{v}_{enc}[T]$ ]
# is available

t = -1
 $\mathbf{h}_{out}[-1] = 0$       # Initial Decoder hidden state
 $\mathbf{q}[0] = 0$           # Initial query

# Note: begins with a "start of sentence" symbol
# <sos> and <eos> may be identical
 $\mathbf{y}_{out}[0] = \text{<sos>}$ 
do
    t = t+1
    C = compute_context_with_attention( $\mathbf{q}[t]$ , K, V)
     $\mathbf{y}[t], \mathbf{h}_{out}[t], \mathbf{q}[t+1] = \text{RNN\_decode\_step}(\mathbf{h}_{out}[t-1], \mathbf{y}_{out}[t-1], \text{C})$ 
     $\mathbf{y}_{out}[t] = \text{generate}(\mathbf{y}[t])$  # Random, or greedy
until  $\mathbf{y}_{out}[t] == \text{<eos>}$ 
```

Example: English to French Translation

Source Sentence (English)

The cat sat on the mat.

Target Sentence (French)

Le chat s'est assis sur le tapis.

- ▶ At each decoding step, the decoder generates a French word by:
 1. Using the current decoder hidden state.
 2. Attending to relevant English words via attention weights.
 3. Computing a context vector as a weighted sum of encoder outputs.
- ▶ For example, when generating “*chat*”, the attention focuses on “cat”.
- ▶ The process repeats for each word, dynamically shifting attention.

- ▶ **Quadratic Complexity:** Attention computation scales as $\mathcal{O}(n^2)$ with sequence length, making it expensive for long sequences.
- ▶ **Shallow Reasoning:** Standard attention mechanisms typically consider only one step of interaction, limiting deep reasoning capabilities.
- ▶ **Noisy Attention Maps:** Attention distributions can be diffuse or hard to interpret, making model decisions less transparent.
- ▶ **Limited Long-Term Memory:** Attention alone may struggle to capture dependencies over very long contexts.

► Efficient Attention Variants:

- *Linformer, Performer, Longformer, FlashAttention* – Reduce memory and computation for long sequences.

► Structured Attention:

- *Sparsemax, Routing Attention* – Impose structure or sparsity for interpretability and efficiency.

► Cross-modal Attention:

- *Vision + Language (e.g., Flamingo, Gato)* – Integrate information across different modalities.

► Learnable Routing:

- *Mixture-of-Experts, Dynamic Attention* – Dynamically select computation paths for scalability.

Concept	Key Takeaway
Alignment	Scores relevance between states
QKV	Core mechanism to compute attention
Decoding	Uses attention to generate outputs
Limitation	Costly, hard to scale to long seqs
Future	Efficient, structured, multimodal attention

These slides have been adapted from:

- ▶ Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization. DeepLearning.AI](#)
- ▶ Bhiksha Raj & Rita Singh, [11-785 Introduction to Deep Learning, CMU](#)

- ▶ Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*.
- ▶ Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need*.
- ▶ Raffel, C., Shazeer, N., Roberts, A., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*.
- ▶ Choromanski, K., Likhoshesterov, V., Dohan, D., et al. (2021). *Rethinking Attention with Performers*.
- ▶ Tay, Y., Dehgani, M., Bahri, D., & Metzler, D. (2023). *Efficient Transformers: A Survey*.
- ▶ Google Research Slides: *The Annotated Transformer*.